Module 4:
Advanced Graph Algorithms
(Part-1)

Dr. Nirnay Ghosh

Graph Coloring

- Definitions:

A *coloring* of a simple graph is the assignment of a color to each vertex of the graph so that no two adjacent vertices are assigned the same color.

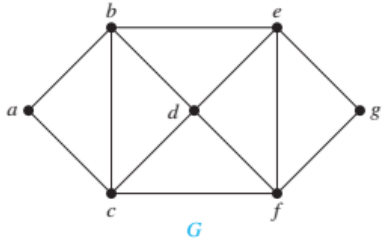
The *chromatic number* of a graph is the least number of colors needed for a coloring of this graph. The chromatic number of a graph G is denoted by $\chi(G)$. (Here χ is the Greek letter *chi*.)

- Finding the chromatic number of a planar graph is equivalent to find the minimum number of colors required to color a planar map so that no two adjacent regions are assigned the same color.
- This question has been studied for more than 100 years.
- The answer is provided by one of the most famous theorems in mathematics.

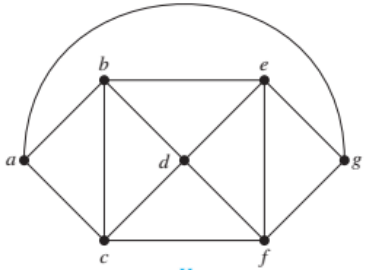
THE FOUR COLOR THEOREM The chromatic number of a planar graph is no greater than four.

Graph Coloring (*Contd...*)

- What are the chromatic numbers of the following graphs?

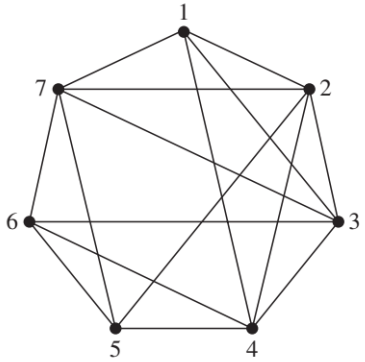


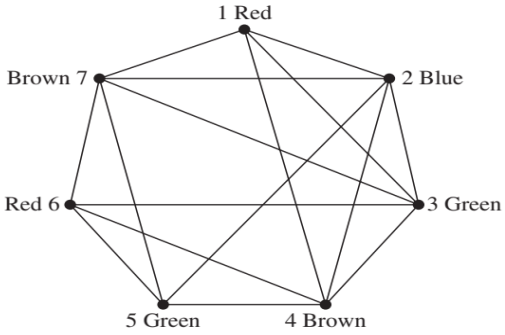
G



H

- What is the chromatic number of K_n?
- What is the chromatic number of the complete bipartite graph $K_{m,n}$, where m and n are positive integers?
- What is the chromatic number of the graph C_n, where $n \geq 3$? (Recall that C_n is the cycle with n vertices.)
- Applications of graph colorings
  - Graph coloring has a variety of applications to problems involving *scheduling* and *assignments*.





Time Period	Courses
I	1, 6
II	2
III	3, 5
IV	4, 7

/2026

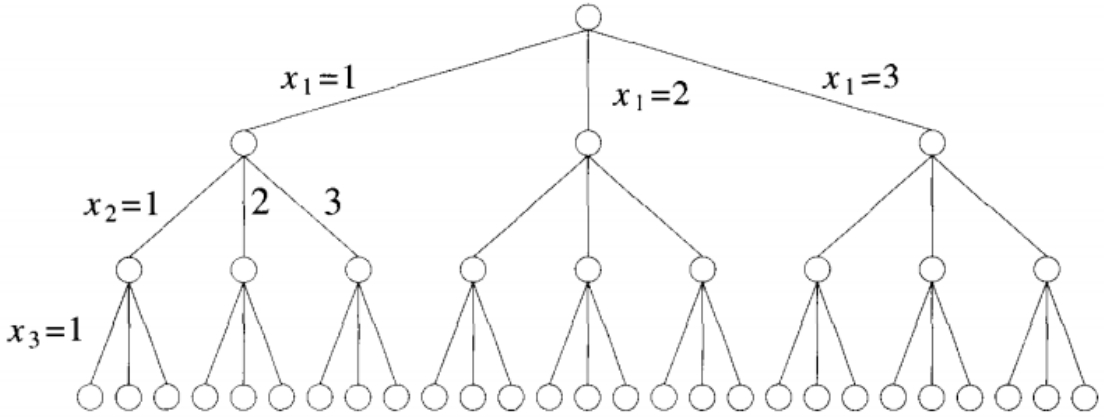
Graph Coloring (*Contd...*)

- A university has to schedule the final examinations for the following eight courses:CS101, CS102, CS201, CS202, CS301, CS302, CS401, CS402
- Two courses cannot be scheduled in the same time slot if there is at least one student enrolled in both courses.
- The following pairs of courses have common students: (CS101, CS102),(CS101, CS201),(CS101, CS301),(CS102, CS201),(CS102, CS202),(CS102, CS302),(CS201, CS202),(CS201, CS301),(CS201, CS401),(CS202, CS301),(CS202, CS402),(CS301, CS302),(CS301, CS401),(CS302, CS402),(CS401, CS402).
- The university wants to use the minimum possible number of examination time slots.
- *Hint: Represent each course by a vertex. Connect two vertices if at least one student is enrolled in both courses. Each color represents one examination time slot.*

Graph Coloring Algorithm

```
1   Algorithm mColoring(k)
2   // This algorithm was formed using the recursive backtracking
3   // schema. The graph is represented by its boolean adjacency
4   // matrix G[1 : n, 1 : n]. All assignments of 1, 2, ..., m to the
5   // vertices of the graph such that adjacent vertices are
6   // assigned distinct integers are printed. k is the index
7   // of the next vertex to color.
8   {
9       repeat
10      {// Generate all legal assignments for x[k].
11          NextValue(k); // Assign to x[k] a legal color.
12          if (x[k] = 0) then return; // No new color possible
13          if (k = n) then     // At most m colors have been
14                              // used to color the n vertices.
15              write (x[1 : n]);
16          else mColoring(k + 1);
17      } until (false);
18  }
```

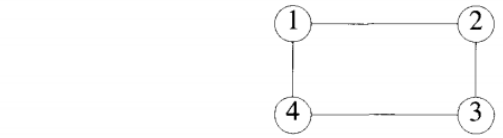
- Nonplanar graphs can have arbitrarily large chromatic number
- Two things are required to show that the chromatic number of a graph is *m*.
  - Show that the graph can be colored with *m* colors.
  - Show that the graph cannot be colored using fewer than *m* colors

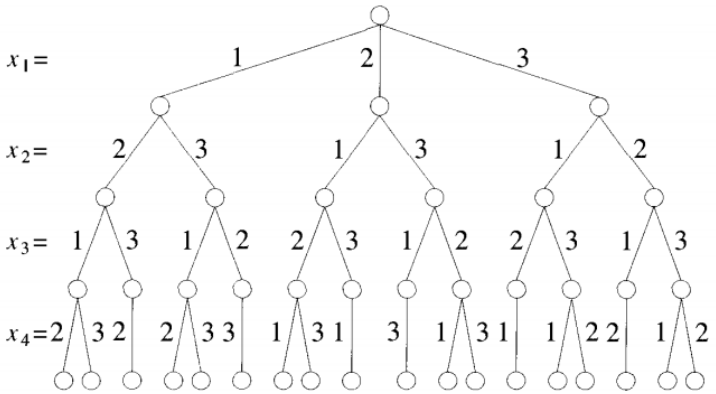


State space tree for mColoring when n = 3 and m = 3

Graph Coloring Algorithm

```
1   Algorithm NextValue(k)
2   // x[1], ..., x[k − 1] have been assigned integer values in
3   // the range [1, m] such that adjacent vertices have distinct
4   // integers. A value for x[k] is determined in the range
5   // [0, m]. x[k] is assigned the next highest numbered color
6   // while maintaining distinctness from the adjacent vertices
7   // of vertex k. If no such color exists, then x[k] is 0.
8   {
9       repeat
10      {
11          x[k] := (x[k] + 1) mod (m + 1); // Next highest color.
12          if (x[k] = 0) then return; // All colors have been used.
13          for j := 1 to n do
14          {   // Check if this color is
15              // distinct from adjacent colors.
16              if ((G[k, j] ≠ 0) and (x[k] = x[j]))
17              // If (k, j) is and edge and if adj.
18              // vertices have the same color.
19                  then break;
20          }
21          if (j = n + 1) then return; // New color found
22      } until (false); // Otherwise try to find another color.
23  }
```





A 4-node Graph and all possible 3-coloring